CronJob

Цели работы:

– изучить концепцию CronJob в Kubernetes для управления периодическими задачами.
– освоить создание и настройку CronJob с использованием расписания в формате Cron.
– понять разницу между Job и CronJob.
– изучить поведение CronJob при накладке выполнения задач (concurrency policies).
– освоить настройку параметров выполнения для отдельных задач Job, создаваемых CronJob (activeDeadlineSeconds, parallelism, completions).

`Cronjob` - выполнение `Job` по определенному графику с определенной периодичностью.

Параметры CronJob

```
spec:
  schedule: "0 2 * * *"        # Расписание (Cron format)
  concurrencyPolicy: "Forbid"   # Политика параллелизма
  startingDeadlineSeconds: 300  # Максимальная задержка запуска
  successfulJobsHistoryLimit: 3 # Хранить 3 успешные Jobs
  failedJobsHistoryLimit: 1     # Хранить 1 неуспешную Job
  jobTemplate:
    spec:
      completions: 6           # Задачи в Job
      parallelism: 2           # Параллельные Pod'ы
```

расписание в Cron-формате ("* * * * *" - минута, час, день, месяц, день недели)

Пример 1. Базовая CronJob

Создаётся простейшая CronJob, которая запускается каждую минуту и демонстрирует основной принцип работы планировщика периодических задач.

Шаг 1: Создаем манифест CronJob

basic-cronjob.yaml – Блокнот

Файл Правка Формат Вид Справка

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: hello-cron
spec:
  schedule: "* * * * *"  # Каждую минуту
  jobTemplate:
    spec:
      template:
        spec:
          containers:
          - name: hello
            image: busybox
            imagePullPolicy: IfNotPresent
            command:
            - /bin/sh
            - -c
            - echo "Hello from CronJob at $(date)"
          restartPolicy: OnFailure
```

Шаг 2: Создаем CronJob и отслеживаем ее работу

- применяем манифест
- проверяем состояние CronJob
- наблюдаем создаваемые Jobs (*в отдельном терминале*)
- через 1-2 минуты проверяем созданные Pods
- смотрим логи любого из созданных Pods

```
C:\Temp>kubectl apply -f basic-cronjob.yaml
cronjob.batch/hello-cron created

C:\Temp>kubectl get cronjob hello-cron
NAME         SCHEDULE    TIMEZONE   SUSPEND   ACTIVE   LAST SCHEDULE   AGE
hello-cron   * * * * *   <none>     False     0        <none>          6s

C:\Temp>kubectl get jobs --watch
NAME                   STATUS    COMPLETIONS   DURATION   AGE
hello-cron-29384468    Running   0/1           2s         2s
hello-cron-29384468    Running   0/1           5s         5s
hello-cron-29384468    Complete   1/1           5s         5s

C:\Temp>kubectl get pods
NAME                        READY   STATUS      RESTARTS        AGE
app-secret-env-pod          1/1     Running     2 (6d23h ago)   14d
app-secret-volume-pod       1/1     Running     2 (6d23h ago)   14d
hello-cron-29384468-cxv7n   0/1     Completed   0               47s

C:\Temp>kubectl logs hello-cron-29384468-cxv7n
Hello from CronJob at Thu Nov 13 21:08:01 UTC 2024

C:\Temp>kubectl get pods
NAME                        READY   STATUS      RESTARTS        AGE
app-secret-env-pod          1/1     Running     2 (6d23h ago)   14d
app-secret-volume-pod       1/1     Running     2 (6d23h ago)   14d
hello-cron-29384468-cxv7n   0/1     Completed   0               2m47s
hello-cron-29384469-ndbq5   0/1     Completed   0               107s
hello-cron-29384470-tghts   0/1     Completed   0               47s
```

Ожидаемый результат:

- каждую минуту создается новая Job
- каждая Job создает Pod, который выполняет команду echo
- в логах видно сообщение "Hello from CronJob at <время>"

Пример 2. CronJob с обработкой ошибок и ограничением времени

Продемонстрируем использование activeDeadlineSeconds для ограничения времени выполнения задачи и поведение CronJob при сбоях.

Шаг 1. Создаем CronJob с ограничением времени выполнения

deadline-cronjob.yaml – Блокнот

Файл Правка Формат Вид Справка

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: limited-task-cron
spec:
  schedule: "*/2 * * * *"  # Каждые 2 минуты
  jobTemplate:
    spec:
      activeDeadlineSeconds: 30  # Максимум 30 секунд на выполнение
      backoffLimit: 2
      template:
        spec:
          containers:
          - name: sleeper
            image: busybox
            command:
            - /bin/sh
            - -c
            - |
              echo "Начинаю долгую операцию..."
              sleep 60
              echo "Операция завершена"
          restartPolicy: OnFailure
```

Шаг 2. Наблюдаем поведение при превышении лимита времени

Создаем CronJob

- отслеживаем создаваемые Jobs
- через 2 минуты проверяем детали созданной Job
- смотрим события, связанные с Job

```
C:\Temp>kubectl apply -f deadline-cronjob.yaml
cronjob.batch/limited-task-cron created

C:\Temp>kubectl get jobs --watch
NAME                          STATUS    COMPLETIONS   DURATION   AGE
limited-task-cron-29384484    Running   0/1                      0s
limited-task-cron-29384484    Running   0/1           0s         0s
limited-task-cron-29384484    Running   0/1           6s         6s
limited-task-cron-29384484    FailureTarget   0/1           30s        30s
limited-task-cron-29384484    FailureTarget   0/1           30s        30s
limited-task-cron-29384484    FailureTarget   0/1           61s        61s
limited-task-cron-29384484    Failed          0/1           61s        61s
limited-task-cron-29384486    Running         0/1                      0s
limited-task-cron-29384486    Running         0/1           0s         0s
limited-task-cron-29384486    Running         0/1           5s         5s
```

```
C:\Temp>kubectl describe job limited-task-cron-29384486
Name:                     limited-task-cron-29384486
Namespace:                default
Selector:                 batch.kubernetes.io/controller-uid=38889ad0-0d32-
Labels:                   batch.kubernetes.io/controller-uid=38889ad0-0d32-
                          batch.kubernetes.io/job-name=limited-task-cron-29
                          controller-uid=38889ad0-0d32-4dc8-b686-3fde29d4f1
                          job-name=limited-task-cron-29384486
Annotations:              batch.kubernetes.io/cronjob-scheduled-timestamp:
Controlled By:            CronJob/limited-task-cron
Parallelism:              1
Completions:              1
Completion Mode:          NonIndexed
Suspend:                  false
Backoff Limit:            2
```

```
C:\Temp>kubectl get events --field-selector involvedObject.name=limited-task-cron-29384486
LAST SEEN   TYPE      REASON            OBJECT                          MESSAGE
2m44s       Normal    SuccessfulCreate  job/limited-task-cron-29384486  Created pod: limited-task-cron-29384486-2nmxl
2m14s       Normal    SuccessfulDelete  job/limited-task-cron-29384486  Deleted pod: limited-task-cron-29384486-2nmxl
103s        Warning   DeadlineExceeded  job/limited-task-cron-29384486  Job was active longer than specified deadline
```

Ожидаемый результат:

- каждые 2 минуты создается новая Job
- каждая Job создает Pod, который "засыпает" на 60 секунд
- через 30 секунд Job принудительно завершается с ошибкой
- в событиях видно сообщение "Job was active longer than requested deadline"

Пример 3/ CronJob с параллельным выполнением и политиками конкурентности

Покажем настройку параллельного выполнения задач и различные политики поведения при наложении запусков.

Шаг 1. Создаем CronJob с параллельными задачами

parallel-cronjob.yaml – Блокнот

Файл Правка Формат Вид Справка

```yaml
kind: CronJob
metadata:
  name: parallel-batch-cron
spec:
  schedule: "*/3 * * * *"  # Каждые 3 минуты
  concurrencyPolicy: Forbid  # Запрещаем параллельные запуски
  jobTemplate:
    spec:
      completions: 6    # Всего должно завершиться 6 Pod'ов
      parallelism: 2    # Одновременно запущено 2 Pod'а
      template:
        spec:
          containers:
          - name: worker
            image: busybox
            command:
            - /bin/sh
            - -c
            - |
              echo "Обрабатываю задачу $HOSTNAME"
              sleep 90
              echo "Задача $HOSTNAME завершена"
          restartPolicy: OnFailure
```

Шаг 2. Наблюдаем параллельное выполнение и работу политики Forbid

- создаем CronJob
- отслеживаем состояние CronJob и создаваемых Jobs
- наблюдаем за Pod'ами в реальном времени
- через 3 минуты проверяем, создалась ли новая Job

```
C:\Temp>kubectl apply -f parallel-cronjob.yaml
cronjob.batch/parallel-batch-cron unchanged

C:\Temp>kubectl get cronjob parallel-batch-cron
NAME                  SCHEDULE      TIMEZONE   SUSPEND   ACTIVE   LAST SCHEDULE   AGE
parallel-batch-cron   */3 * * * *   <none>     False     1        35s             70s

C:\Temp>kubectl get jobs
NAME                            STATUS    COMPLETIONS   DURATION   AGE
parallel-batch-cron-29384499    Running   0/6           43s        43s

C:\Temp>kubectl get pods --watch
NAME                                READY   STATUS    RESTARTS     AGE
app-secret-env-pod                  1/1     Running   2 (7d ago)   14d
app-secret-volume-pod               1/1     Running   2 (7d ago)   14d
parallel-batch-cron-29384499-2sbr4  1/1     Running   0            62s
parallel-batch-cron-29384499-jmvfw  1/1     Running   0            62s
parallel-batch-cron-29384499-jmvfw  0/1     Completed   0            94s
parallel-batch-cron-29384499-2sbr4  0/1     Completed   0            95s
parallel-batch-cron-29384499-jmvfw  0/1     Completed   0            96s
parallel-batch-cron-29384499-txxwv  0/1     Pending     0            0s
parallel-batch-cron-29384499-txxwv  0/1     Pending     0            0s
parallel-batch-cron-29384499-txxwv  0/1     ContainerCreating   0            0s
parallel-batch-cron-29384499-jmvfw  0/1     Completed           0            96s
parallel-batch-cron-29384499-2sbr4  0/1     Completed           0            97s
parallel-batch-cron-29384499-k76x9  0/1     Pending             0            0s
parallel-batch-cron-29384499-k76x9  0/1     Pending             0            0s
parallel-batch-cron-29384499-2sbr4  0/1     Completed           0            98s
parallel-batch-cron-29384499-k76x9  0/1     ContainerCreating   0            2s
parallel-batch-cron-29384499-txxwv  1/1     Running             0            4s
```

```
C:\Temp>kubectl get jobs
NAME                            STATUS    COMPLETIONS   DURATION   AGE
parallel-batch-cron-29384499    Running   2/6           3m13s      3m13s

C:\Temp>kubectl get jobs
NAME                            STATUS    COMPLETIONS   DURATION   AGE
parallel-batch-cron-29384499    Running   4/6           3m34s      3m34s

C:\Temp>kubectl get cronjob parallel-batch-cron
NAME                  SCHEDULE      TIMEZONE   SUSPEND   ACTIVE   LAST SCHEDULE   AGE
parallel-batch-cron   */3 * * * *   <none>     False     1        4m16s           4m51s
```

Что видно из вывода:

- Job parallel-batch-cron-29384499 продолжает работать (STATUS = Running)
- COMPLETIONS: увеличивается с 2/6 до 4/6 за время наблюдения
- LAST SCHEDULE: 4m16s - это означает, что 4 минуты 16 секунд назад CronJob пыталась создать новую Job
- ACTIVE: 1 - указывает на активную Job

Что произошло (работа политики Forbid):

- Время 0:00 - Создана Job parallel-batch-cron-29384499
- Время 3:00 - Наступило время нового запуска CronJob
- CronJob проверяет: "Есть ли активные Jobs?"
- Обнаружена работающая Job parallel-batch-cron-29384499
- Политика `Forbid` запрещает параллельные запуски
- Запуск ПРОПУЩЕН - новая Job не создана

Шаг 3. Тестируем разные политики конкурентности

Меняем политику в манифесте:

`concurrencyPolicy: Allow` # Разрешаем параллельные запуски (по умолчанию)

```
C:\Temp>kubectl apply -f parallel-cronjob.yaml
cronjob.batch/parallel-batch-cron configured

C:\Temp>kubectl get cronjob parallel-batch-cron
NAME                  SCHEDULE      TIMEZONE   SUSPEND   ACTIVE   LAST SCHEDULE   AGE
parallel-batch-cron   */3 * * * *   <none>     False     2        112s            8m27s

C:\Temp>kubectl get pods --watch
NAME                                    READY   STATUS      RESTARTS     AGE
app-secret-env-pod                      1/1     Running     2 (7d ago)   14d
app-secret-volume-pod                   1/1     Running     2 (7d ago)   14d
parallel-batch-cron-29384499-2sbr4      0/1     Completed   0            8m2s
parallel-batch-cron-29384499-hxjzq      0/1     Completed   0            4m48s
parallel-batch-cron-29384499-jmvfw      0/1     Completed   0            8m2s
parallel-batch-cron-29384499-k76x9      0/1     Completed   0            6m24s
parallel-batch-cron-29384499-rfmhn      0/1     Completed   0            4m49s
parallel-batch-cron-29384499-txxwv      0/1     Completed   0            6m26s
parallel-batch-cron-29384502-5zm25      1/1     Running     0            90s
parallel-batch-cron-29384502-dk4pv      1/1     Running     0            92s
parallel-batch-cron-29384502-p7mpp      0/1     Completed   0            3m10s
parallel-batch-cron-29384502-z2r4s      0/1     Completed   0            3m10s
parallel-batch-cron-29384505-fg7lg      1/1     Running     0            24s
parallel-batch-cron-29384505-p4ffz      1/1     Running     0            24s
parallel-batch-cron-29384502-dk4pv      0/1     Completed   0            95s
parallel-batch-cron-29384502-5zm25      0/1     Completed   0            94s
parallel-batch-cron-29384502-dk4pv      0/1     Completed   0            97s
parallel-batch-cron-29384502-7gdmt      0/1     Pending     0            0s
parallel-batch-cron-29384502-7gdmt      0/1     Pending     0            0s
parallel-batch-cron-29384502-dk4pv      0/1     Completed   0            97s
parallel-batch-cron-29384502-7gdmt      0/1     ContainerCreating   0            0s
parallel-batch-cron-29384502-5zm25      0/1     Completed           0            96s
parallel-batch-cron-29384502-wsgqg      0/1     Pending             0            0s
```

```
C:\Temp>kubectl get jobs
NAME                          STATUS     COMPLETIONS   DURATION   AGE
parallel-batch-cron-29384499  Complete   6/6           4m52s      8m25s
parallel-batch-cron-29384502  Running    4/6           3m33s      3m33s
parallel-batch-cron-29384505  Running    0/6           47s        48s

C:\Temp>kubectl get cronjob parallel-batch-cron
NAME                  SCHEDULE      TIMEZONE   SUSPEND   ACTIVE   LAST SCHEDULE   AGE
parallel-batch-cron   */3 * * * *   <none>     False     2        2m48s           9m23s
```

Что видно из вывода:

- три Jobs одновременно: 29384499 (Complete), 29384502 (Running), 29384505 (Running)
- ACTIVE: 2 - две активные Jobs
- LAST SCHEDULE: 2m48s - последний запуск был 2 минуты 48 секунд назад
- COMPLETIONS: прогресс выполнения - 6/6, 4/6, 0/6

Хронология событий:

- Время 0:00 - Создана Job parallel-batch-cron-29384499

- Время 3:00 - Создана Job parallel-batch-cron-29384502 (политика Allow)
- Время 6:00 - Создана Job parallel-batch-cron-29384505 (политика Allow)
- Время 8:25 - Job 29384499 завершилась (6/6 Complete)
- Время 9:23 - Текущее состояние: две Jobs еще работают

```
C:\Temp>kubectl get jobs
NAME                          STATUS     COMPLETIONS   DURATION   AGE
parallel-batch-cron-29384499  Complete   6/6           4m52s      10m
parallel-batch-cron-29384502  Complete   6/6           4m56s      5m21s
parallel-batch-cron-29384505  Running    2/6           2m35s      2m36s
parallel-batch-cron-29384508  Running    0/6           73s        73s

C:\Temp>kubectl get cronjob parallel-batch-cron
NAME                  SCHEDULE      TIMEZONE   SUSPEND   ACTIVE   LAST SCHEDULE   AGE
parallel-batch-cron   */3 * * * *   <none>     False     2        78s             10m
```

`concurrencyPolicy: Replace` # Заменяем предыдущую незавершенную Job

```
C:\Temp>kubectl apply -f parallel-cronjob.yaml
cronjob.batch/parallel-batch-cron configured

C:\Temp>kubectl get cronjob parallel-batch-cron
NAME                  SCHEDULE      TIMEZONE   SUSPEND   ACTIVE   LAST SCHEDULE   AGE
parallel-batch-cron   */3 * * * *   <none>     False     1        2m57s           15m
```

```
C:\Temp>kubectl get pods --watch
NAME                                    READY   STATUS      RESTARTS   AGE
parallel-batch-cron-29384511-hcdt4      0/1     Completed   0          96s
parallel-batch-cron-29384511-hcdt4      0/1     Completed   0          96s
parallel-batch-cron-29384511-hcdt4      0/1     Completed   0          96s
parallel-batch-cron-29384511-hnqnw      0/1     Completed   0          96s
parallel-batch-cron-29384511-hnqnw      0/1     Completed   0          96s
parallel-batch-cron-29384511-hnqnw      0/1     Completed   0          96s
```

```
C:\Temp>kubectl get jobs
NAME                          STATUS     COMPLETIONS   DURATION   AGE
parallel-batch-cron-29384502  Complete   6/6           4m56s      10m
parallel-batch-cron-29384505  Complete   6/6           4m56s      7m57s
parallel-batch-cron-29384508  Complete   6/6           4m54s      6m34s
parallel-batch-cron-29384514  Running    0/6           34s        34s

C:\Temp>kubectl get cronjob parallel-batch-cron
NAME                  SCHEDULE      TIMEZONE   SUSPEND   ACTIVE   LAST SCHEDULE   AGE
parallel-batch-cron   */3 * * * *   <none>     False     1        40s             16m

C:\Temp>kubectl get jobs
NAME                          STATUS     COMPLETIONS   DURATION   AGE
parallel-batch-cron-29384502  Complete   6/6           4m56s      12m
parallel-batch-cron-29384505  Complete   6/6           4m56s      9m29s
parallel-batch-cron-29384508  Complete   6/6           4m54s      8m6s
parallel-batch-cron-29384514  Running    2/6           2m6s       2m6s

C:\Temp>kubectl get cronjob parallel-batch-cron
NAME                  SCHEDULE      TIMEZONE   SUSPEND   ACTIVE   LAST SCHEDULE   AGE
parallel-batch-cron   */3 * * * *   <none>     False     1        2m11s           17m
```

Что видно из вывода:

- 3 завершенные Jobs: 29384502, 29384505, 29384508 (все Complete 6/6)
- 1 активная Job: 29384514 (Running 0/6)
- ACTIVE: 1
- LAST SCHEDULE: 40s

Что произошло (хронология для Replace):

text

- ~10m назад - Создана Job 29384502
- ~7m57s назад - Время нового запуска!
- Replace: УДАЛЕНА 29384502 (незавершенная), создана 29384505
- ~6m34s назад - Время нового запуска!

- Replace: УДАЛЕНА 29384505 (незавершенная), создана 29384508
- ~4m54s назад - Job 29384508 УСПЕЛА завершиться до следующего запуска!
- ~40s назад - Создана Job 29384514 (предыдущая уже завершена - замены не нужно)

Сравнение:

- при политике Forbid: новая Job не создается, если предыдущая еще выполняется
- при политике Allow: новые Jobs создаются по расписанию независимо от статуса предыдущих
- при политике Replace: незавершенная Job удаляется и создается новая
- внутри каждой Job работает 2 параллельных Pod'a до завершения всех 6 задач

Задания для самостоятельного решения

Задание 10.1. CronJob с выводом системной информации
Создайте CronJob с именем `system-info-cron`, которая:
- запускается каждые 5 минут
- выводит информацию о системе: текущее время, имя узла и список процессов
- использует команду: echo "Time: $(date), Host: $(hostname), Processes: $(ps aux | wc -l)"

Проверочные команды:
```
kubectl get cronjob system-info-cron
kubectl get jobs --selector=job-name=system-info-cron
kubectl logs -l job-name=<имя-созданной-job>
```

Задание 10.2. CronJob с контролем ресурсов
Создайте CronJob с именем `resource-checker-cron`, которая:
- запускается каждые 3 минуты
- имеет activeDeadlineSeconds: 45
- выполняет команду, которая иногда превышает лимит: sleep $((random % 60 + 30))
- использует backoffLimit: 2

Проверочные команды:
```
kubectl describe cj resource-checker-cron
kubectl get jobs -w
```

Какие Jobs завершаются успешно, а какие - с ошибкой?

Задание 10.3. CronJob с последовательной обработкой
Создайте CronJob с именем `sequential-processor-cron`, которая:
- запускается каждые 4 минуты
- создает Job, которая обрабатывает 4 задачи (completions: 4) последовательно (parallelism: 1)
- каждая задача выполняется командой: echo "Processing item $((RANDOM % 100))" && sleep 30
- использует политику параллелизма Forbid

Проверочные команды:
```
kubectl get cj sequential-processor-cron -o yaml | grep concurrencyPolicy
kubectl get pods --watch
```

Убедитесь, что задачи выполняются последовательно, а не параллельно

Задание 10.4. CronJob с разными политиками
Создайте CronJob с именем `policy-test-cron`, которая:
- запускается каждые 2 минуты
- выполняет долгую операцию: sleep 150 (2.5 минуты)
- протестируйте три политики конкурентности:
  1) Forbid - пропущенные запуски
  2) Allow - накопление Jobs
  3) Replace - замена незавершенных Jobs

Проверочные команды:

```
kubectl get jobs --sort-by=.metadata.creationTimestamp
kubectl get cronjob policy-test-cron -o wide
```

Сравните поведение системы при разных политиках

Контрольные вопросы

1. В чем основное отличие CronJob от обычной Job?
2. Какой формат используется для задания расписания в CronJob? Что означают 5 полей?
3. Какие три политики параллелизма (concurrency policy) существуют у CronJob и в чем их разница?
4. Что произойдет, если выполнение задачи, инициированной CronJob, превысит activeDeadlineSeconds?
5. Где настраиваются параметры parallelism и completions для задач, создаваемых CronJob?
6. Как можно посмотреть логи конкретной Job, которая была создана CronJob?
7. Что произойдет, если CronJob будет запланирована на запуск в момент, когда кластер Kubernetes недоступен?
8. Какие команды вы бы использовали для диагностики проблем с CronJob, которая не создает Jobs?